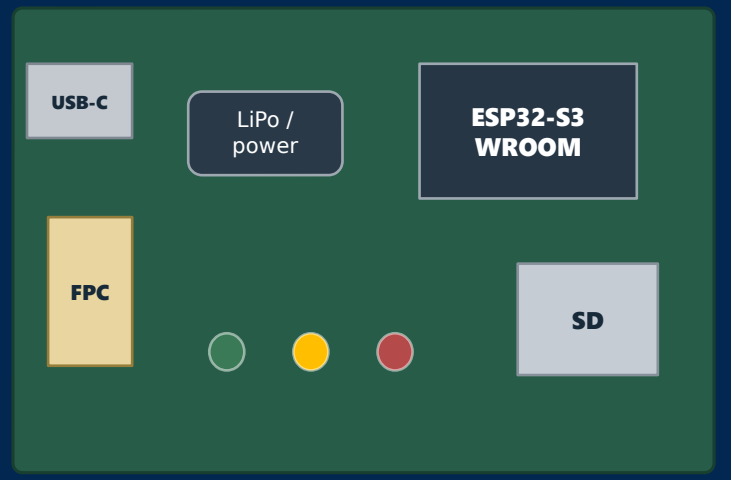
MEMS Microphone Recorder System

PCB development, firmware bring-up, troubleshooting, and next deployment





PDM audio | **local WAV** | **BLE control**

Sam Amezcua | UC Davis | EDIT PRESENTATION DATE

What I was building

A compact audio platform that works without a PC in the recording loop

- Capture PDM MEMS microphone data as standard WAV files
- Store recordings locally for untethered experiments
- Trigger without touching the setup: pushbutton or Bluetooth
- Make failures visible through LEDs, telemetry, and recoverable files
- Support both a quick devkit prototype and custom microphone hardware

PLANNED USE — EDIT THIS LINE

[Replace with the exact final experiment/application.]

Current evidence supports describing it as a portable, remotely triggered acoustic-data platform for controlled or field measurements.

Design principle **Record first, verify continuously, and preserve usable audio when hardware or storage misbehaves.**

Three ordered PCB designs form the hardware path

Prototype quickly, scale the sensor geometry, then integrate the recorder

1 ESP32 devkit carrier

Rapid bring-up board

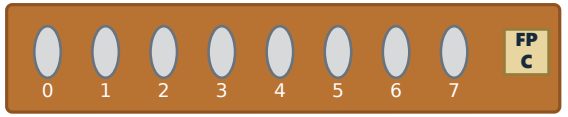
- ESP32-S3 DevKitC
- Two PDM microphones
- microSD + buttons + LEDs
- Platform used for firmware validation



2 Eight-mic flex array

Spatial sensor board

- Eight IMP34DT05TR PDM mics
- Four left/right microphone pairs
- Local decoupling
- 10-pin FPC tail



3 Integrated controller

Portable system direction

- ESP32-S3-WROOM-N8R8
- USB-C + LiPo charging
- 3.3 V and 1.8 V rails
- FPC, SD, buttons, and status LEDs



Board 1 — ESP32 devkit recorder

The fast-learning platform: prove the microphone, storage, controls, and enclosure before full integration



48 kHz | stereo | 16-bit PCM | microSD

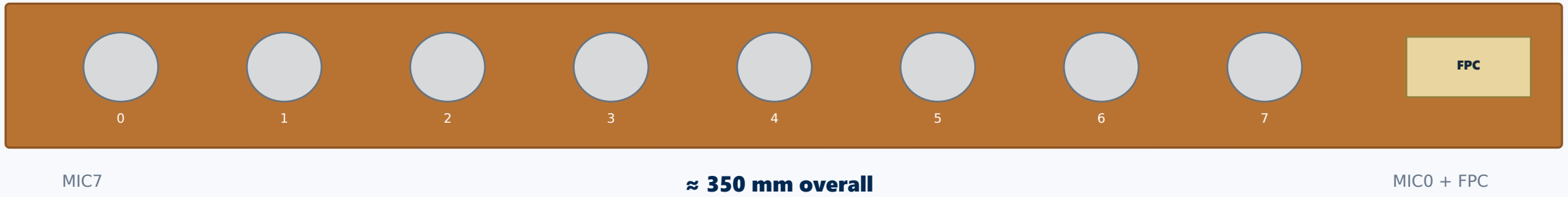
Validated pin map **PDM 4/5 • SD SPI 12/13/11/14 • button 35 • LEDs 40/41/42**

What is on this board

- ESP32-S3 DevKitC module
- Two ST IMP34DT05TR PDM microphones
- 1.8 V microphone regulator
- microSD socket over SPI
- Record button and three status LEDs
- Exported STEP plus printable enclosure files

Board 2 — eight-microphone flex array

A narrow sensor strip for known microphone spacing and multi-point acoustic capture



Sensor layout

Eight ST IMP34DT05TR omnidirectional digital PDM microphones arranged along a narrow flex PCB.

Electrical strategy

Four left/right PDM pairs, shared clocking, paired data behavior, and local 100 nF / 10 µF decoupling.

System role

Moves the microphones away from the controller and enables repeatable spatial placement through one 10-pin FPC interface.

Design status: ordered PCB design; validate each pair, cable integrity, clock loading, and channel mapping during bring-up.

Board 3 — integrated microphone controller

Turns the bench prototype into a battery-capable, connectorized recorder



USB-C + ESD | **LiPo charge / load share** | **FPC sensor interface**

Integration decisions

- ESP32-S3-WROOM-1-N8R8 replaces the plug-in dev board
- MCP73871 battery charging and load sharing
- Dedicated 3.3 V and 1.8 V regulation
- 10-position FPC connector for external mic hardware
- SD interface, USB access, controls, and visible status
- Mechanical mounting holes and compact enclosure path

Engineering shift

From “can this record?” to “can this be deployed, powered, connected, and recovered reliably?”

Firmware evolved in five deliberate steps

Each intermediate build isolated one subsystem before combining them

1. **PDM → serial** — Captured raw PCM to a PC with a Python script.
2. **USB microphone** — Exposed the board as a USB Audio Class device.
3. **SD diagnostic** — Probed SPI behavior and swept mount speeds.
4. **Standalone WAV** — Recorded locally with button control and status LEDs.
5. **BLE + reliability** — Added remote control, telemetry, recovery, and storage guards.

Current recorder: bench validated and working

The key reliability fix was separating capture from storage

SD writes are bursty; microphone sampling cannot wait



Failure observed

A slow SD transaction could exceed the I²S DMA buffering. Samples were silently lost, so playback sounded sped up with clicks and pops.

Result after redesign

The reader runs continuously while storage latency consumes buffer headroom. Average SD throughput comfortably exceeds the 192 kB/s audio stream.

Control plane: button + BLE commands → start/stop | status path: LEDs + 500 ms BLE heartbeat

Troubleshooting that materially changed the design

The project became robust by reproducing failures and adding explicit recovery paths

Observed problem	Root cause	Fix / lesson
Sped-up audio / clicks	SD writes blocked capture	Dedicated core-0 reader + 128 KB queue
SD failed after reset	Card left in partial SPI transaction	Send ≥74 clocks before every mount
Level LED froze with no card	Full mount retry ran every 50 ms	Separate 50 ms probe from 1 s remount
Pulled card ruined whole WAV	Header size updated only at stop	Refresh WAV header every 1 s
Old take overwritten	Flaky remount restarted filename scan	Monotonic in-RAM recording index
No serial / build too large	Board options were wrong	CDCOnBoot=cdc + huge_app partition
Upload port busy	IDE daemon held COM port	Stop the background Arduino CLI process

What the recorder can do now

Bench-tested on the custom ESP32-S3 PCB

AUDIO

48 kHz • stereo • 16-bit PCM

Standard incrementing WAV files

CONTROL

Button or Bluetooth LE

Self-contained Web Bluetooth remote

PROOF

Live byte counter

Confirms data is actually reaching the card

STORAGE

20 MHz SPI on a 32 GB card

200 MB warning / 50 MB hard stop

RECOVERY

Hot-plug inference + periodic headers

At most ~1 s tail loss on card removal

STATUS

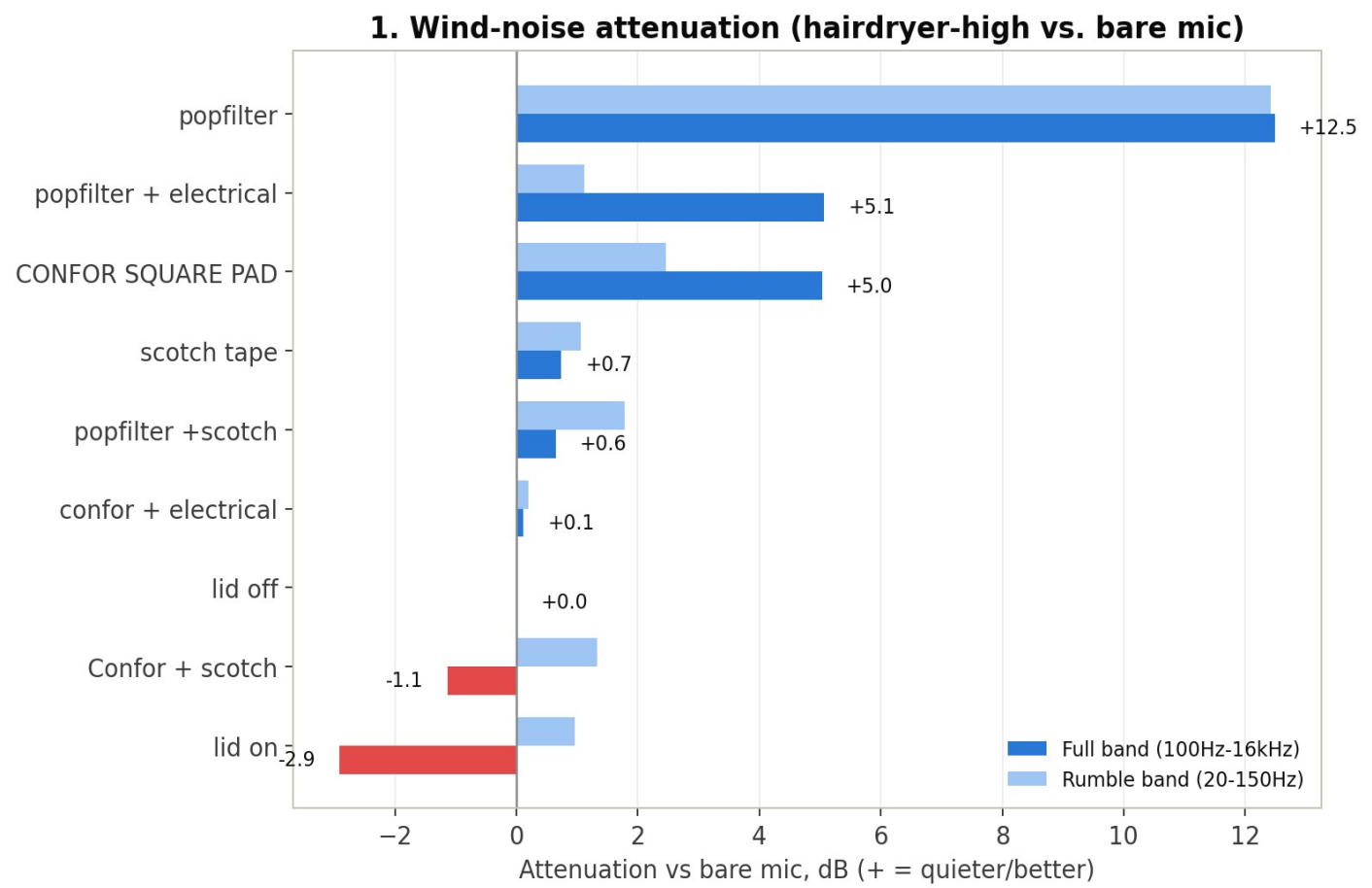
Three LEDs + 500 ms heartbeat

Recording, SD, free space, and error state

Known-good boot: SD mounted at 20 MHz → BLE advertising → ready to record

Material damping check — feasibility only

A quick screen to learn what was possible and verify that the recorder produced analyzable data



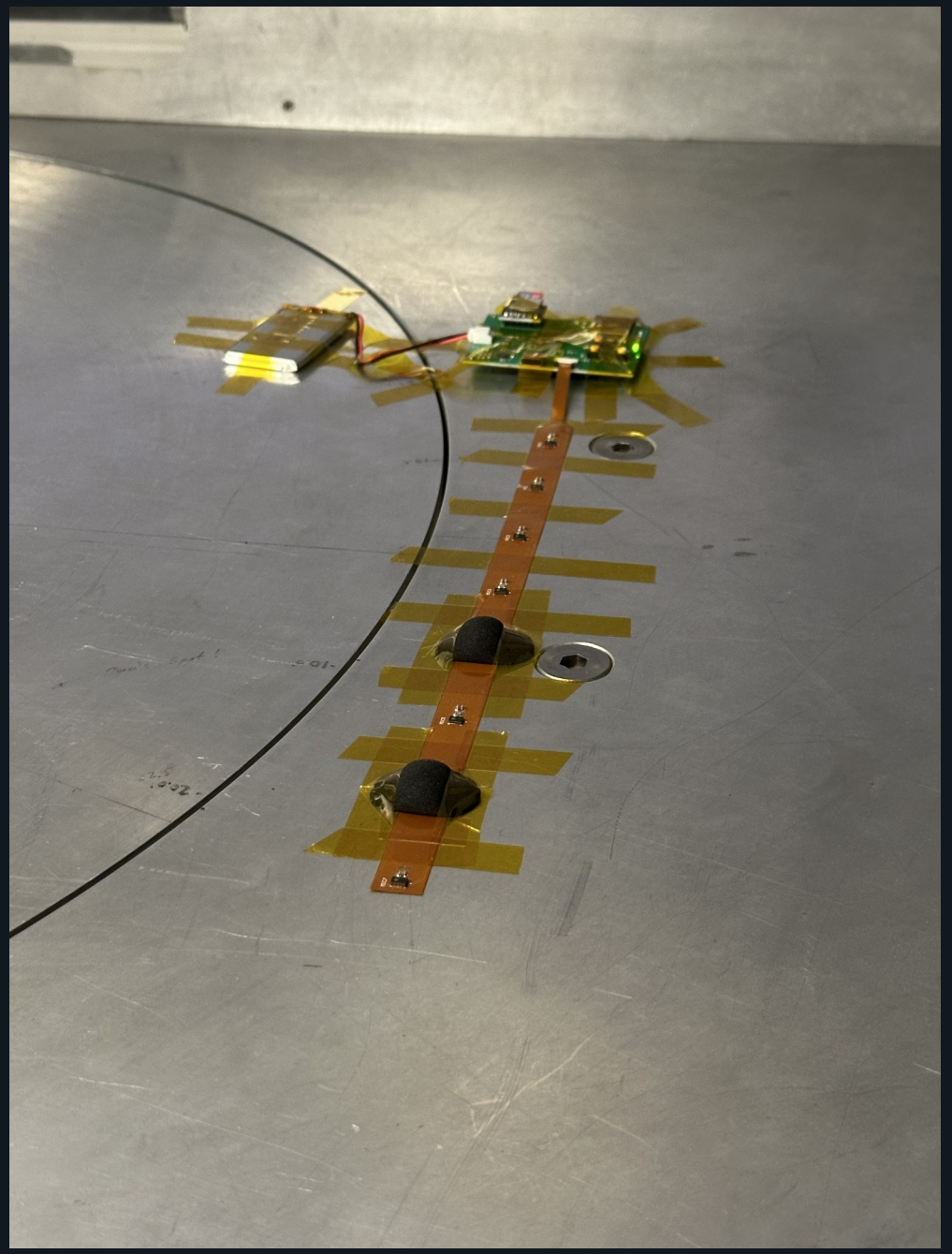
What was done

Nine coverings × ambient / low / high hairdryer conditions. One short recording per combination; the source was hand-positioned.

Only takeaway

Bare pop-filter fabric looked promising (~12.5 dB lower recorded level than lid-off under high airflow). This is not calibrated attenuation or a scientific treatment comparison.

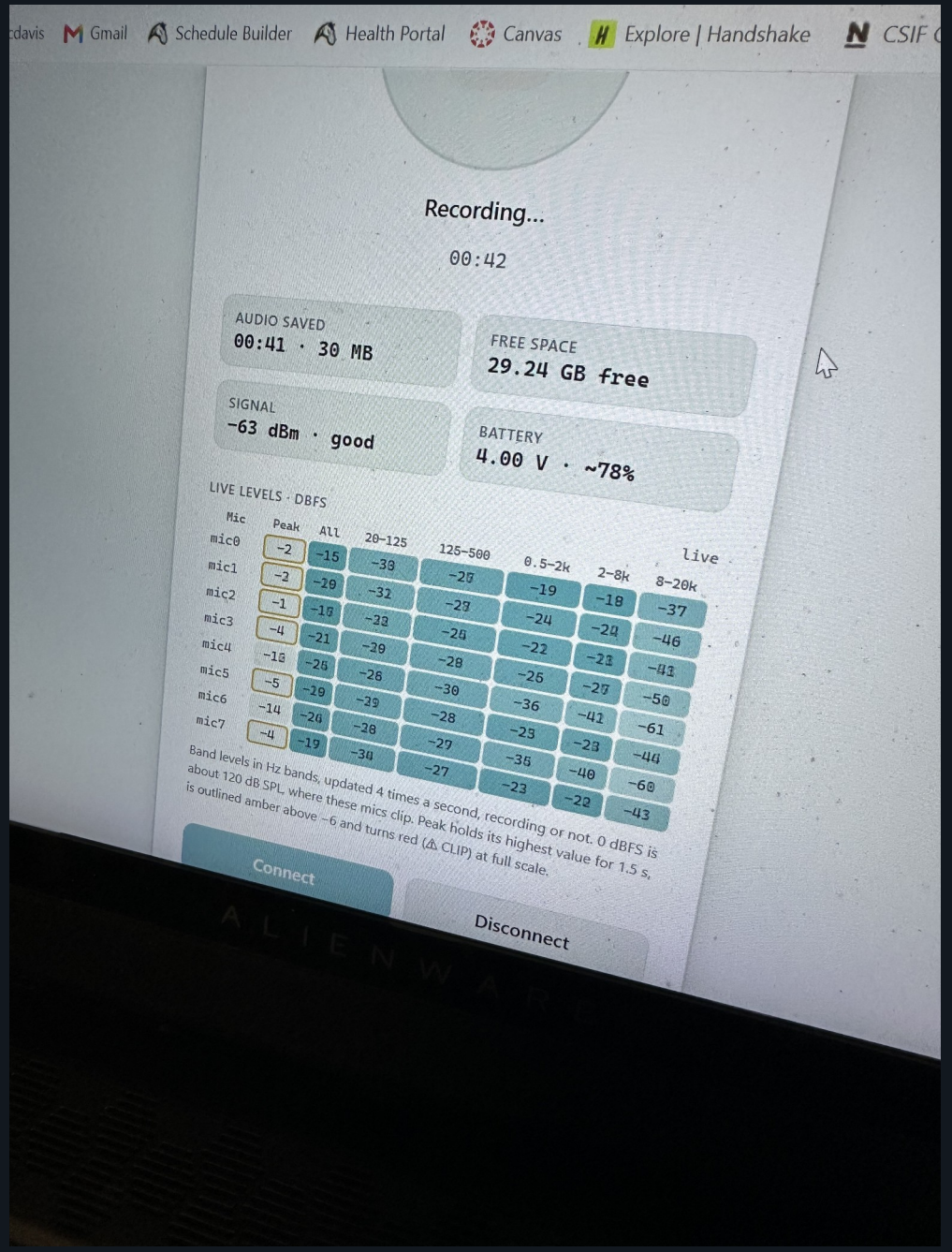
Presentation framing: engineering proof-of-concept and material down-selection—not a validated acoustic experiment.



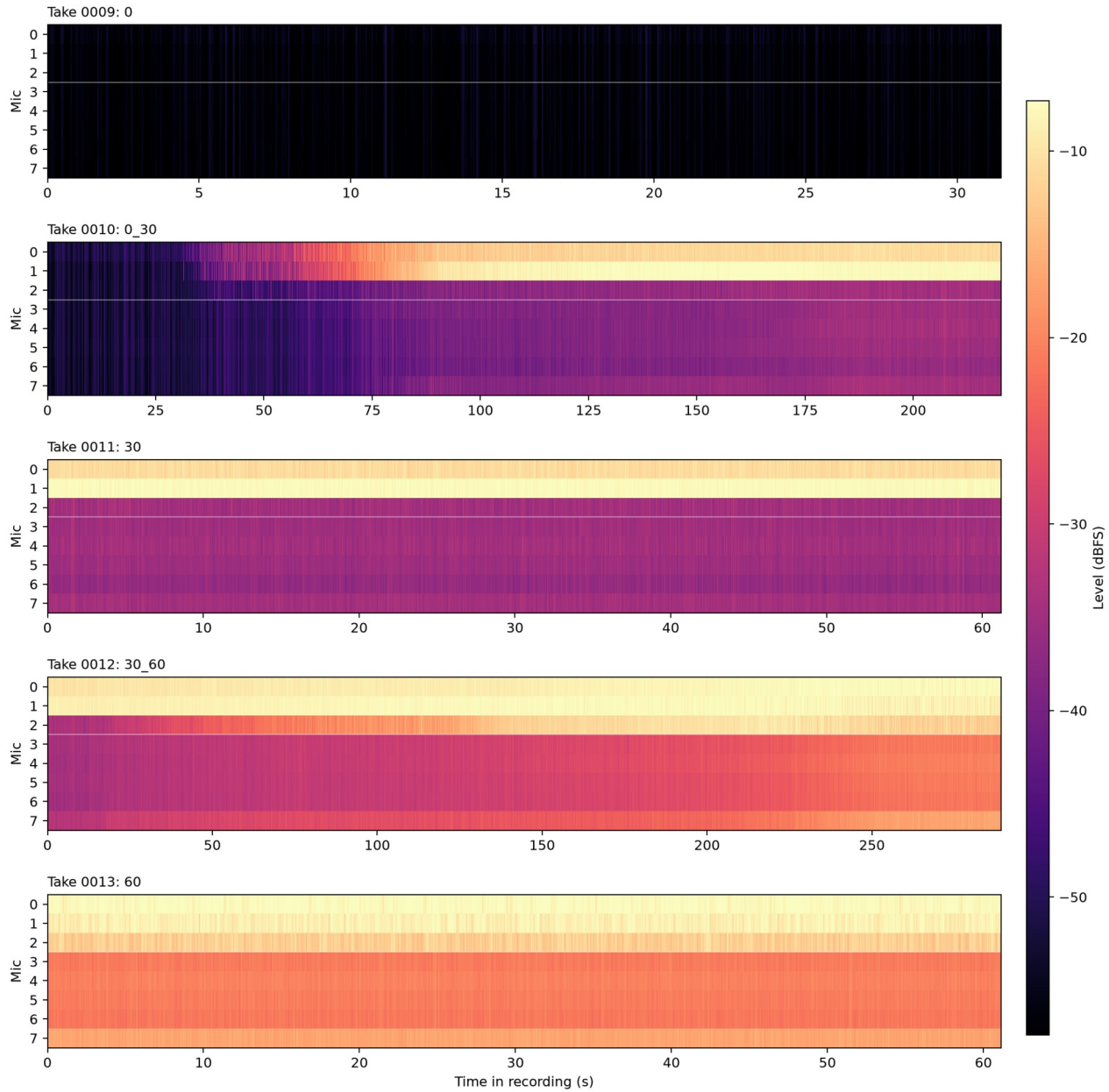
Eight-microphone array with both pop filters taped down

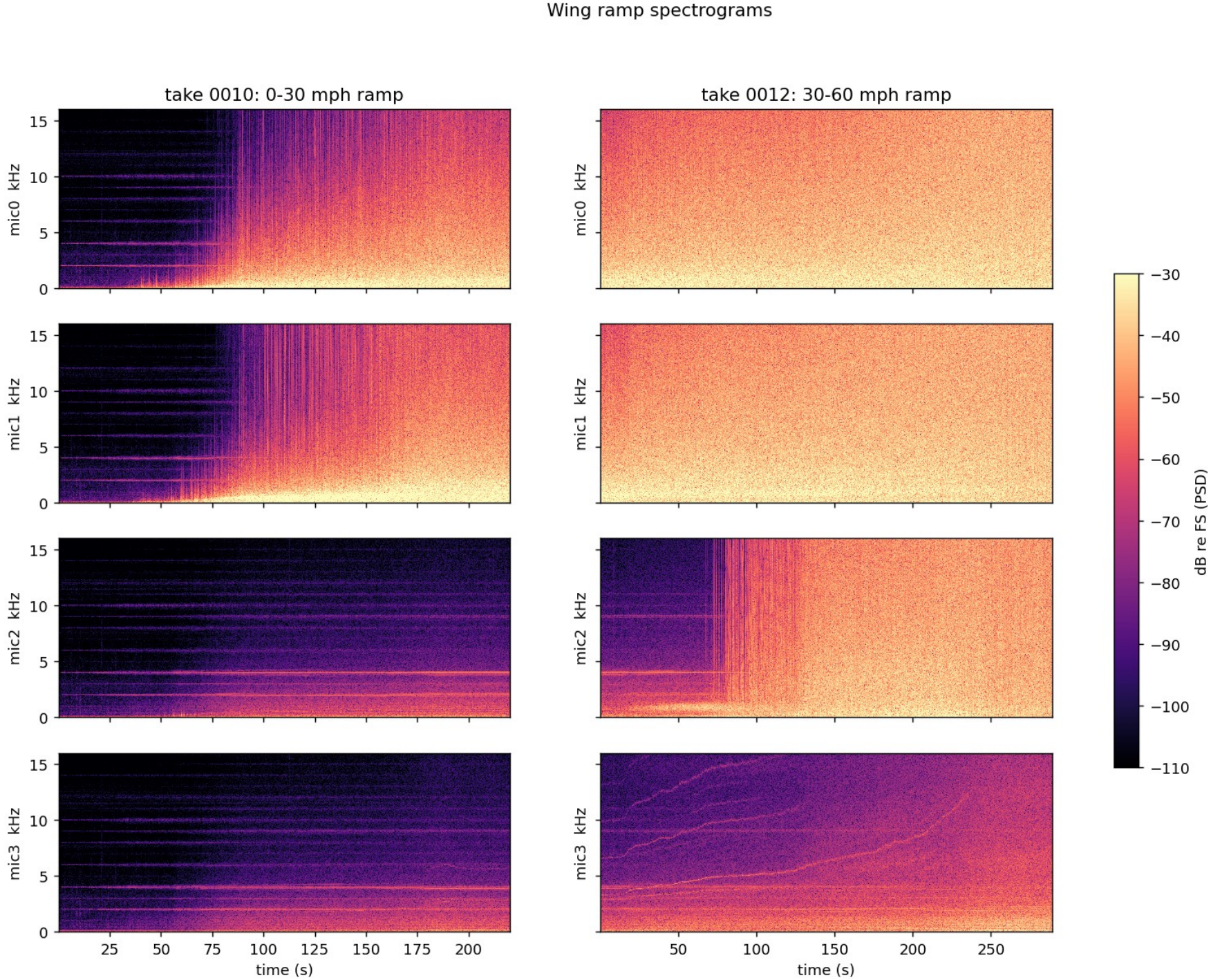


Array installed on the wing

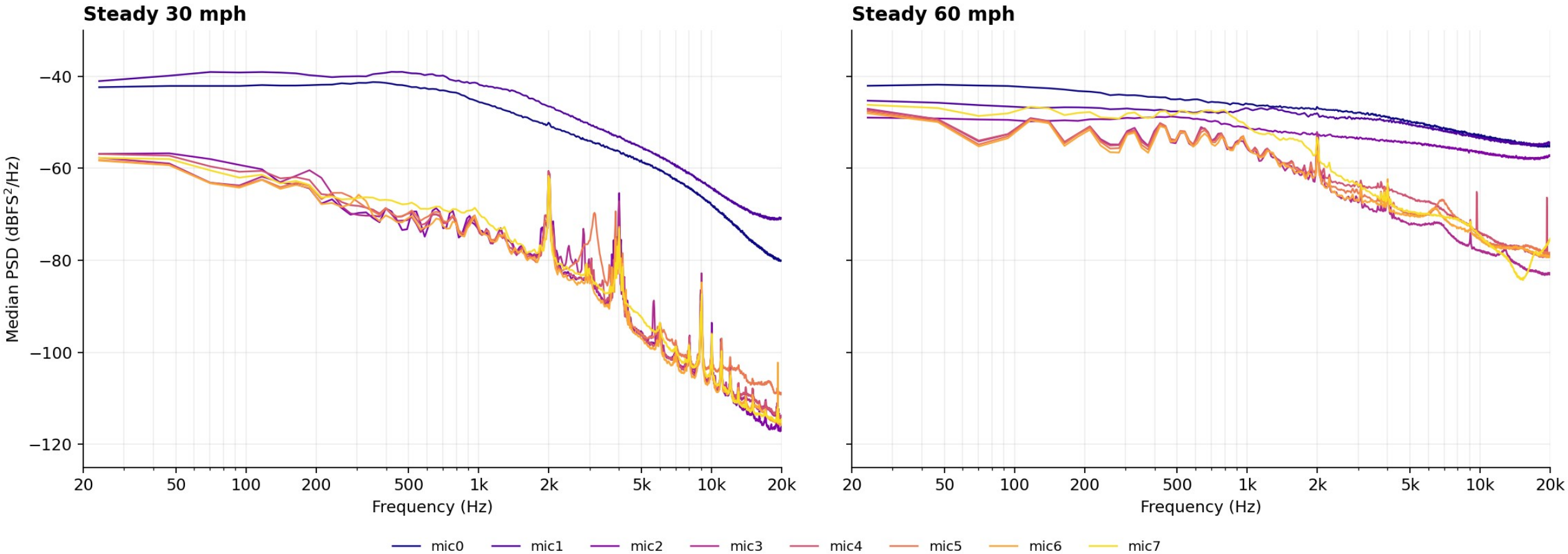


Live eight-channel band-level display during recording


Wing runs: 20 Hz-20 kHz level over time
Take 0009: 0
Take 0010: 0_30
Take 0011: 30
Take 0012: 30_60
Take 0013: 60
Mic
Time in recording (s)
Level (dBFS)
−10
−20
−30
−40
−50


Wing ramp spectrograms
take 0010: 0-30 mph ramp
take 0012: 30-60 mph ramp
mic0 kHz
mic1 kHz
mic2 kHz
mic3 kHz
0
5
10
15
25
50
75
100
125
150
175
200
250
time (s)
−30
−40
−50
−60
−70
−80
−90
−100
−110
dB re FS (PSD)

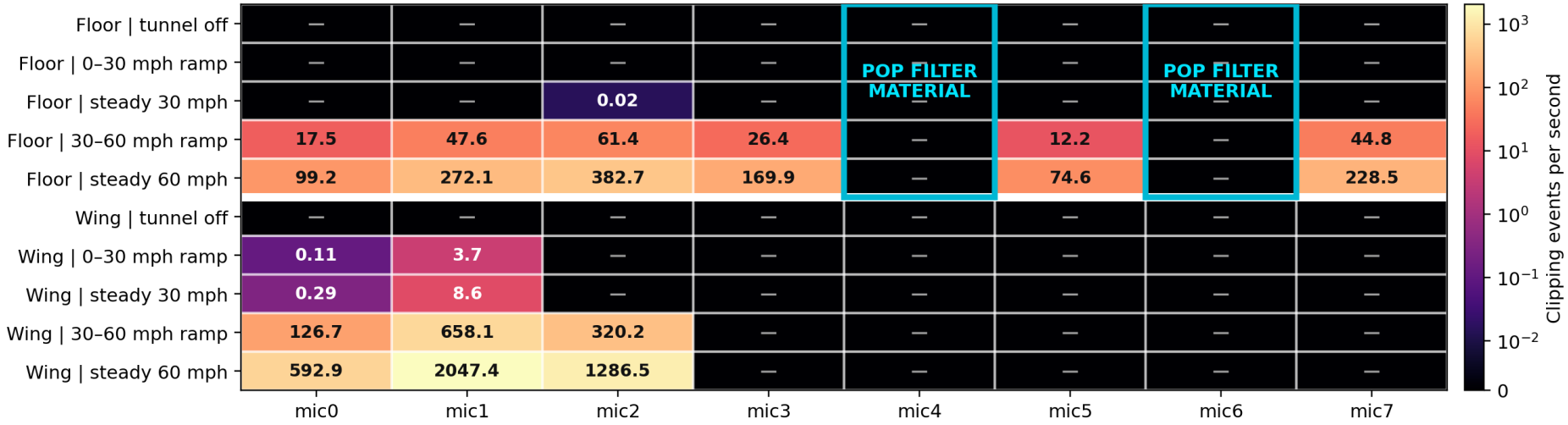
Steady wing broadband power spectral density



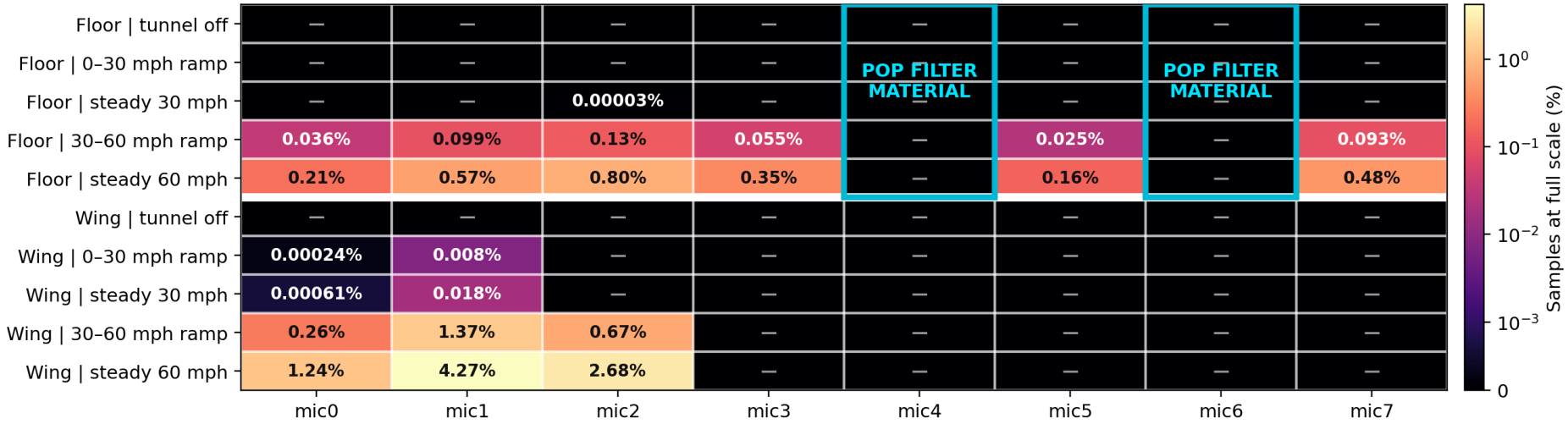
Full-scale clipping by microphone and test condition

Clipping is a raw 16-bit sample at −32768 or +32767; one event is a contiguous run of clipped samples

How often clipping starts: distinct full-scale events per second



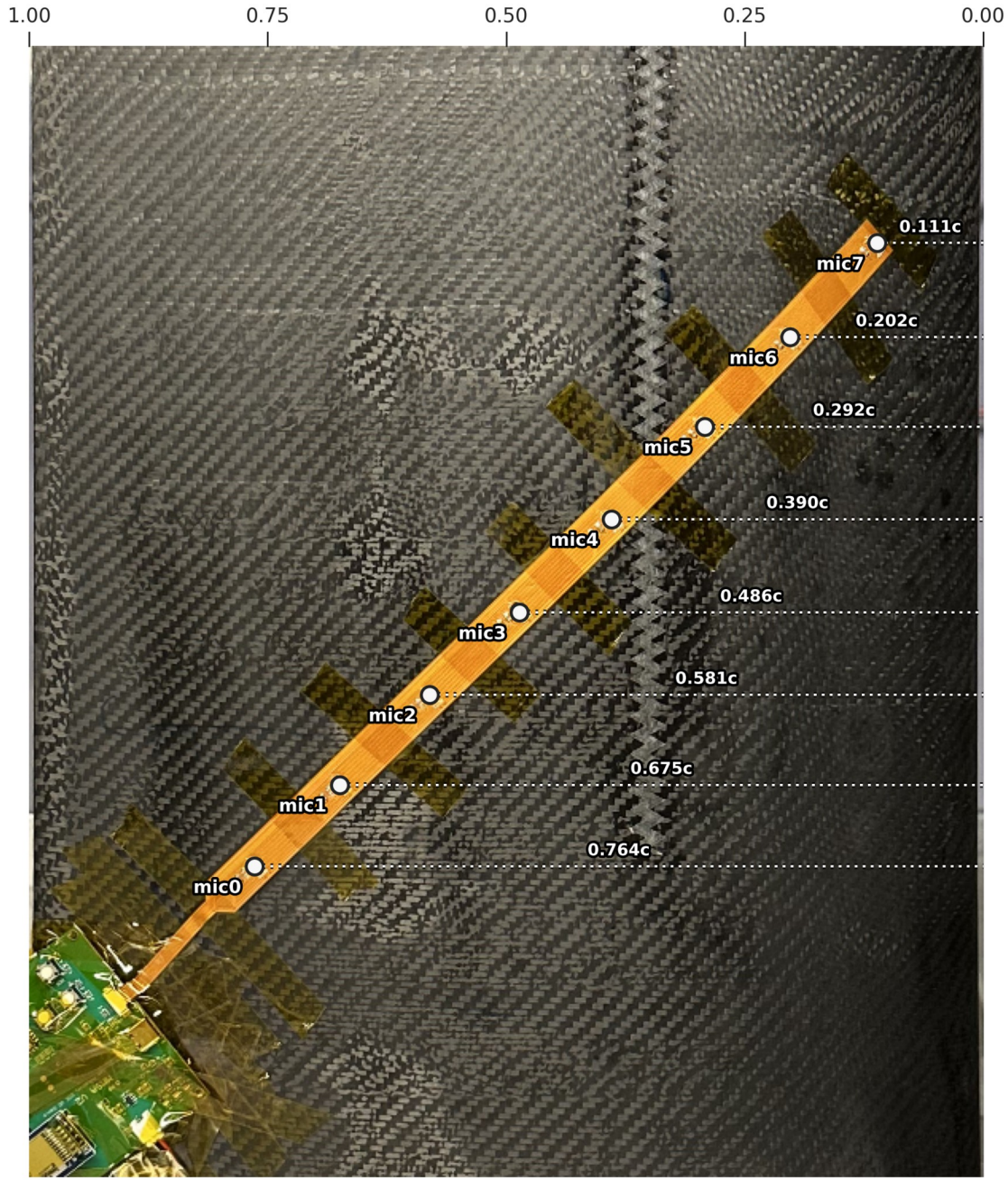
How much data is saturated: samples at full scale



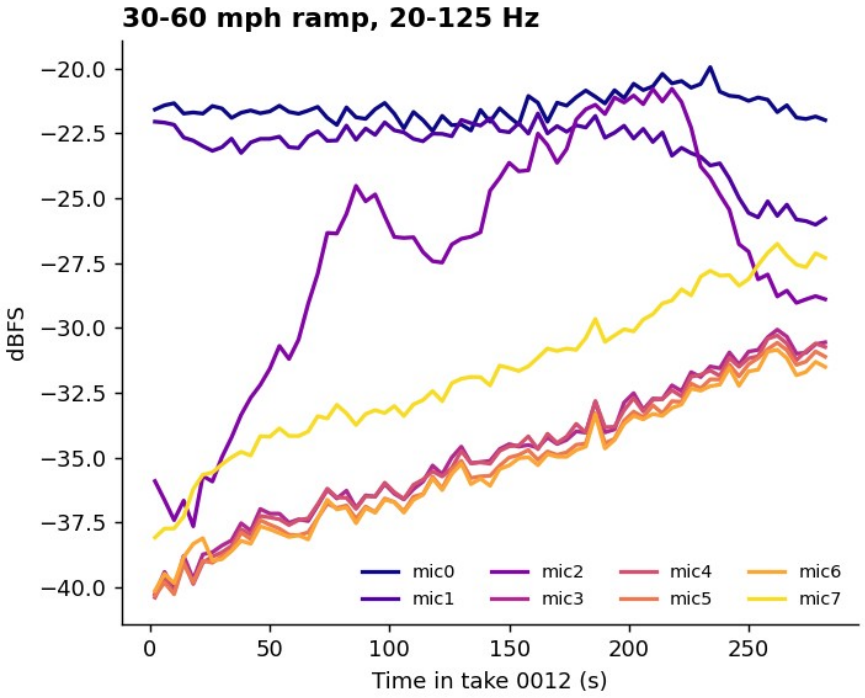
Cyan boxes: floor mic4 and mic6 were covered with pop-filter material, which is why they never reached full scale.

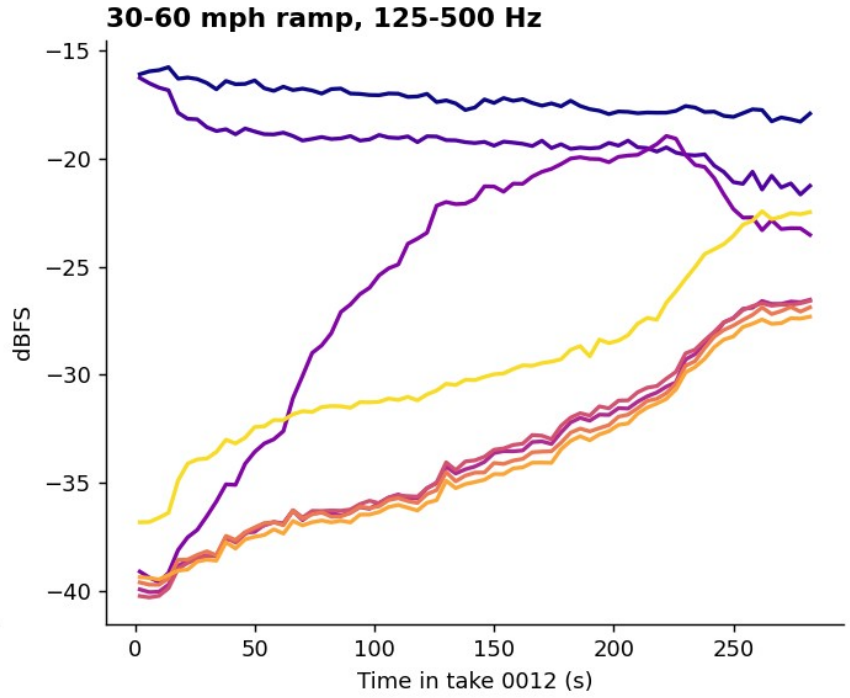
Measured microphone positions on the rectified wing image

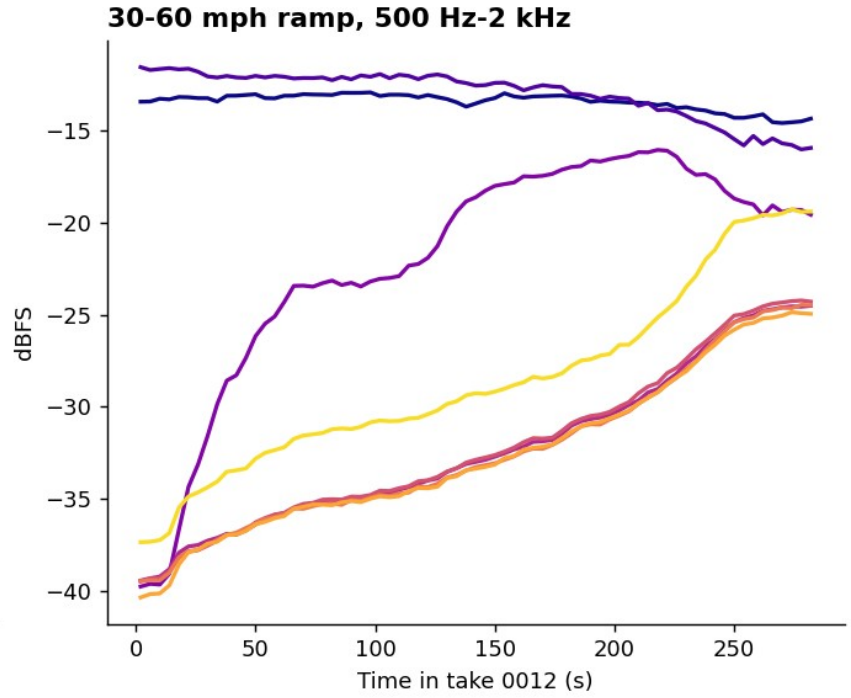
Coordinate used in analysis: x/c from leading edge

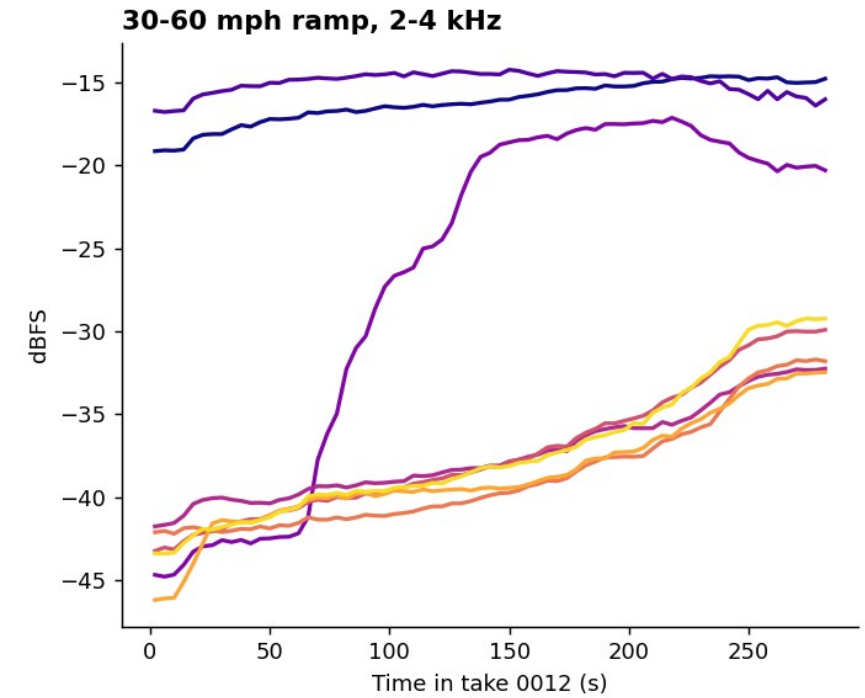


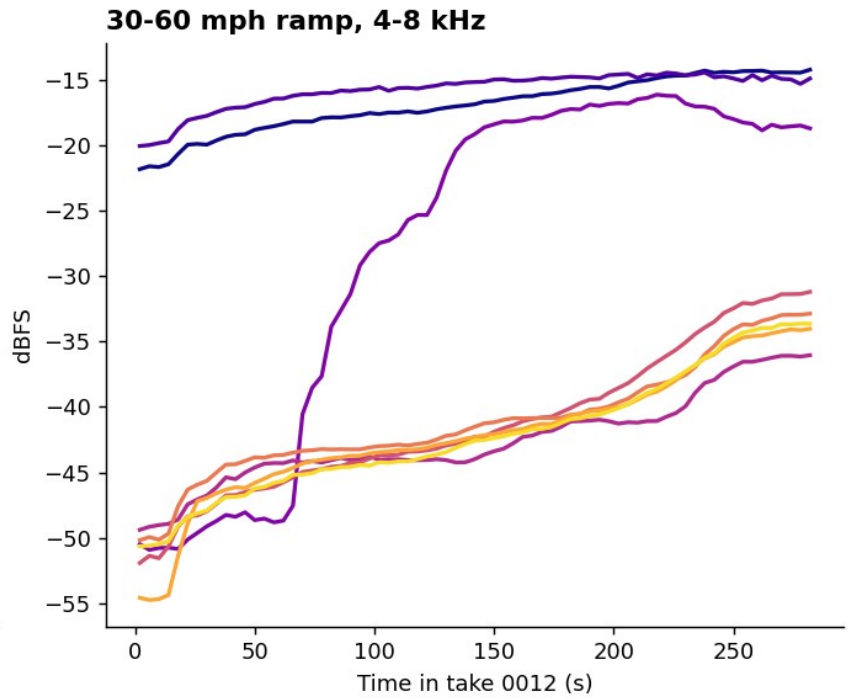
Ramp level per microphone, by frequency band

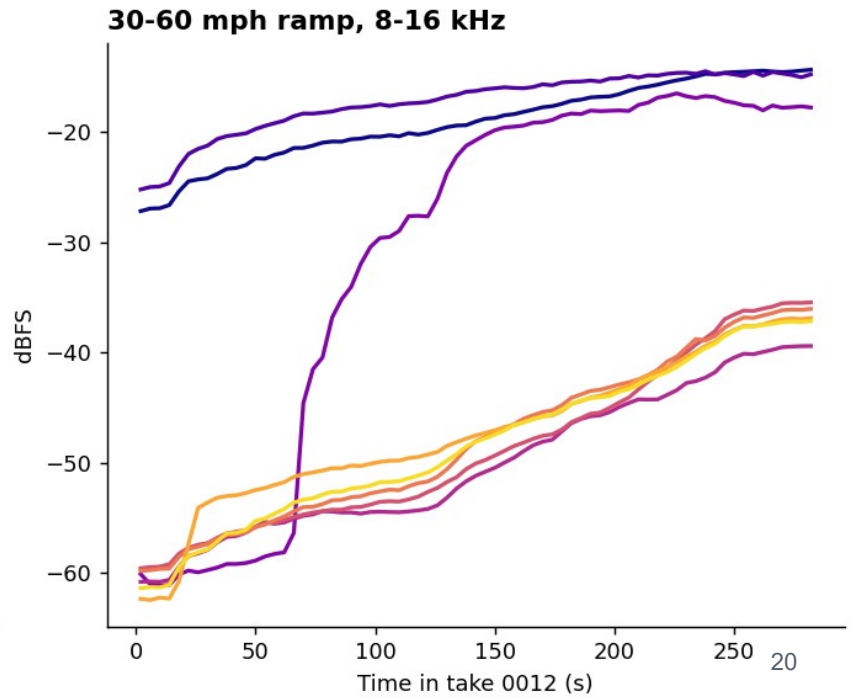




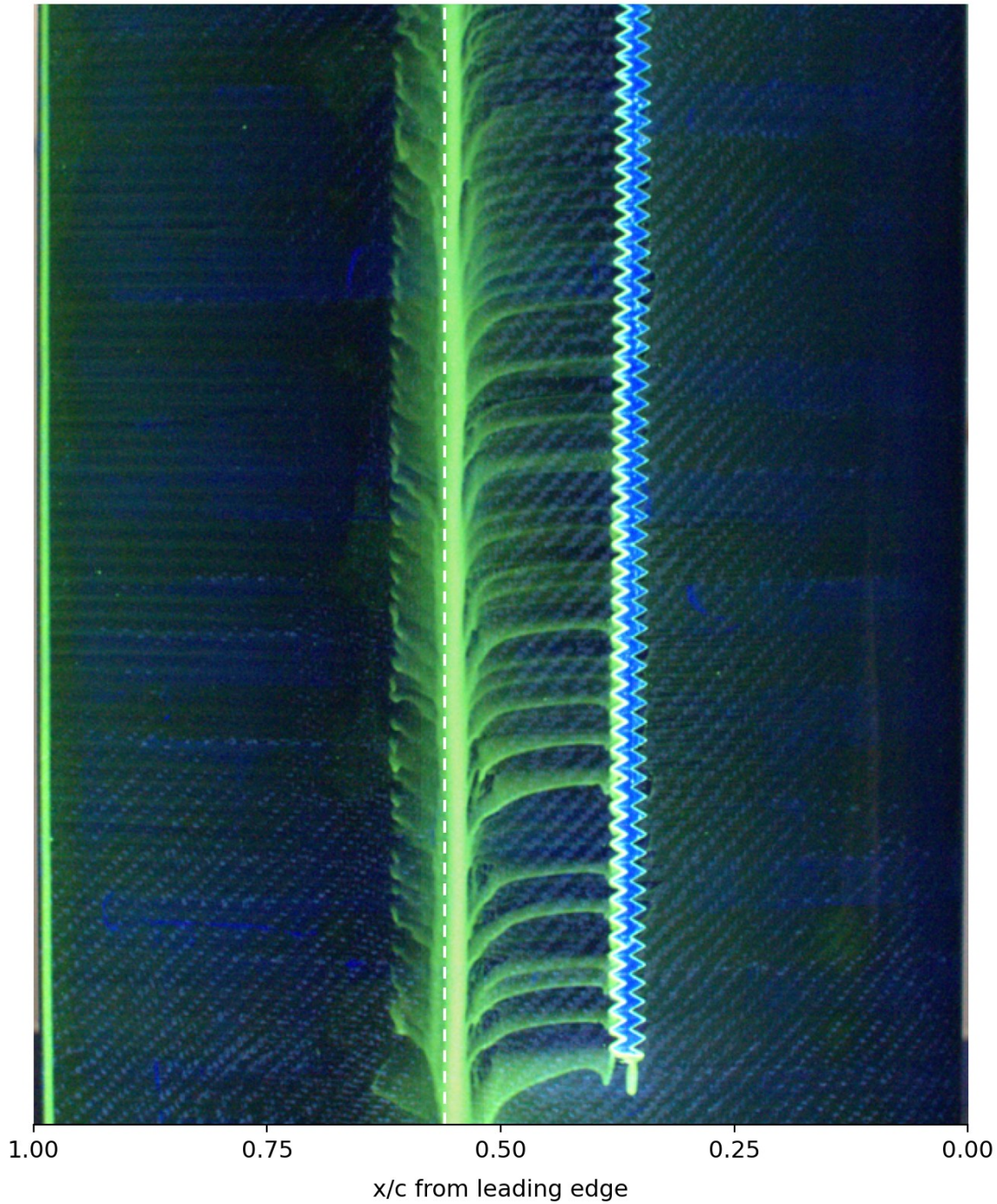




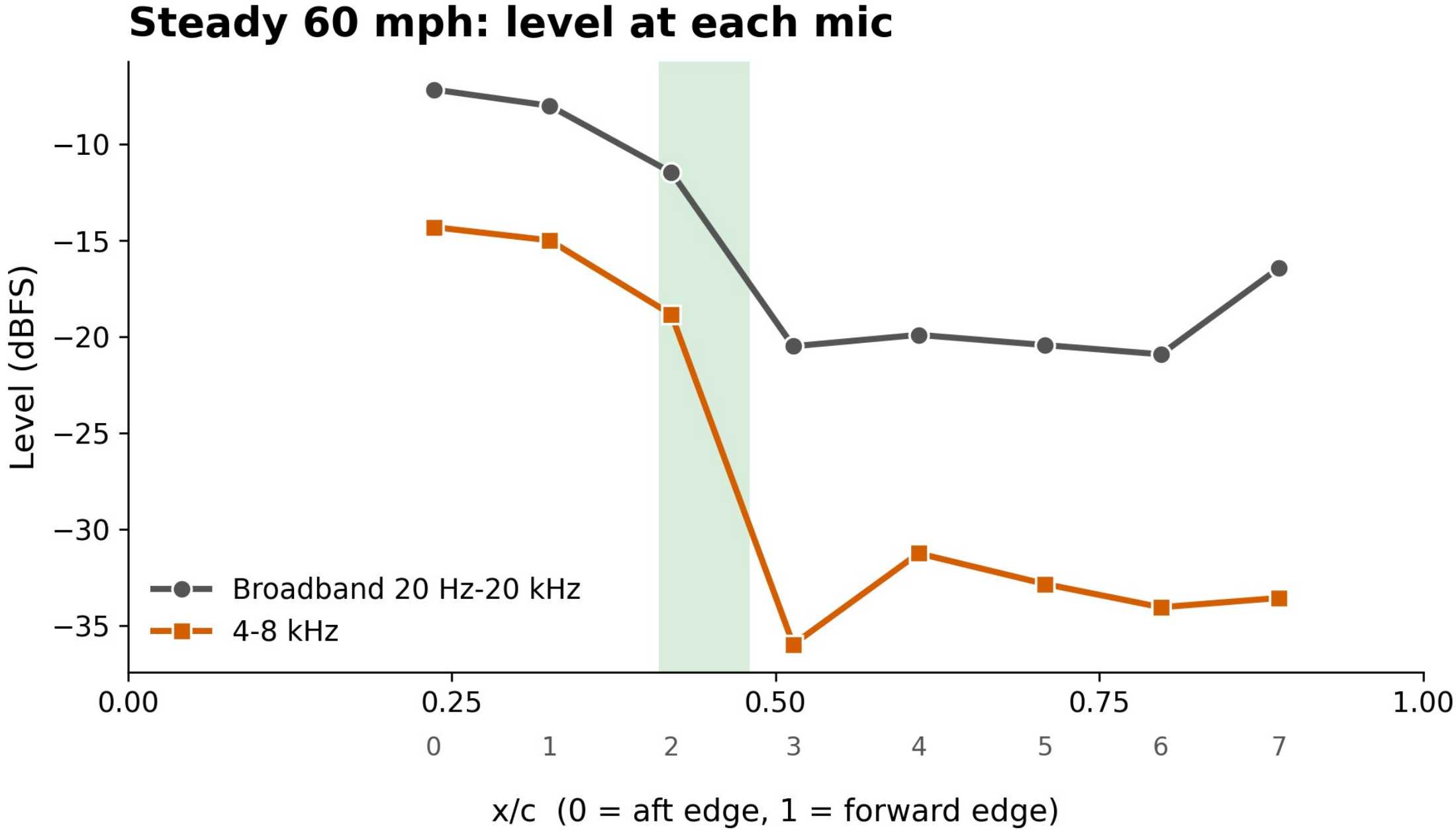


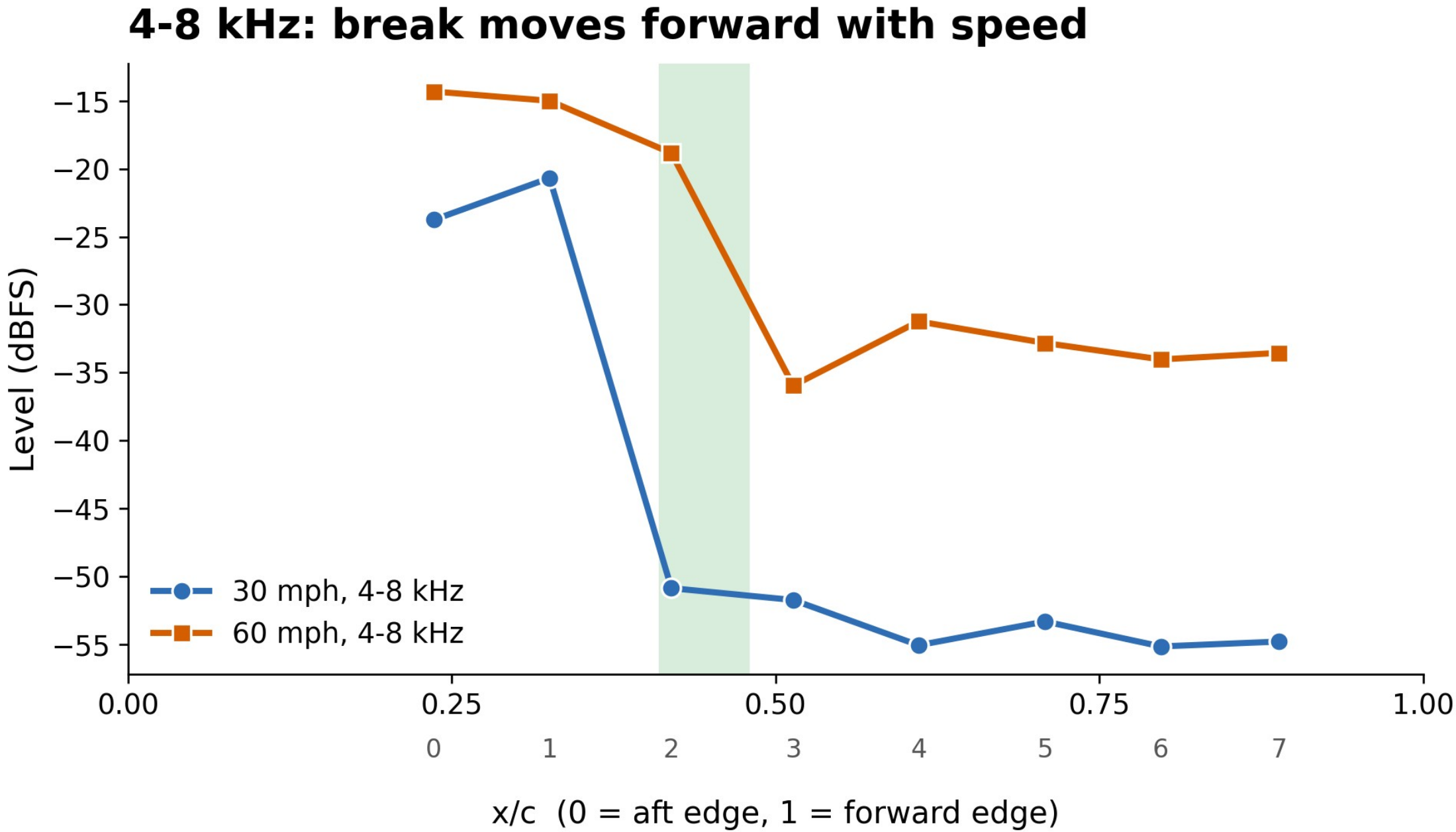


60 mph fluorescent flow visualization



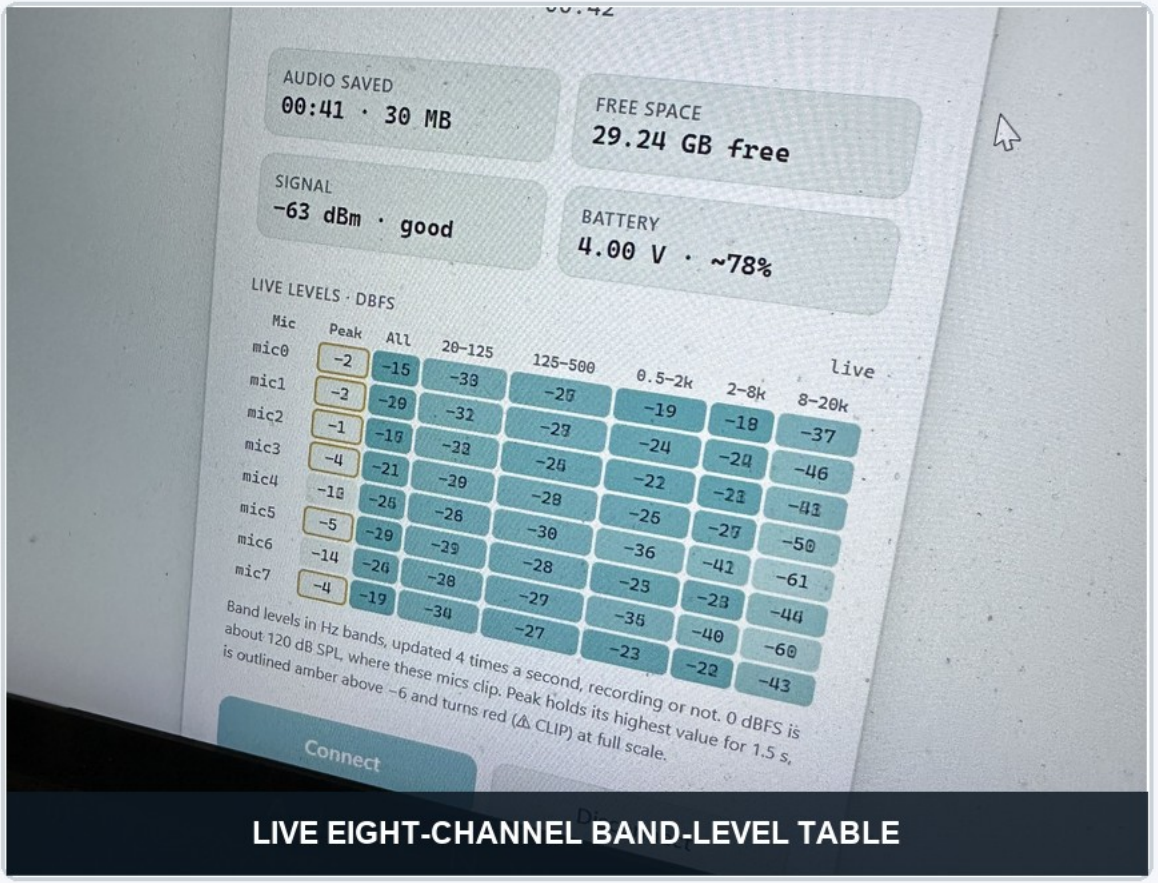

Overlay on the shared chord axis
mic2-mic3
level break
mic7
mic6
mic5
mic4
mic3
mic2
mic1
mic0
0.00
0.25
0.50
0.75
1.00


Steady 60 mph: level at each mic
Level (dBFS)
−10
−15
−20
−25
−30
−35
Broadband 20 Hz-20 kHz
4-8 kHz
0.00
0.25
0.50
0.75
1.00
0
1
2
3
4
5
6
7
x/c (0 = aft edge, 1 = forward edge)

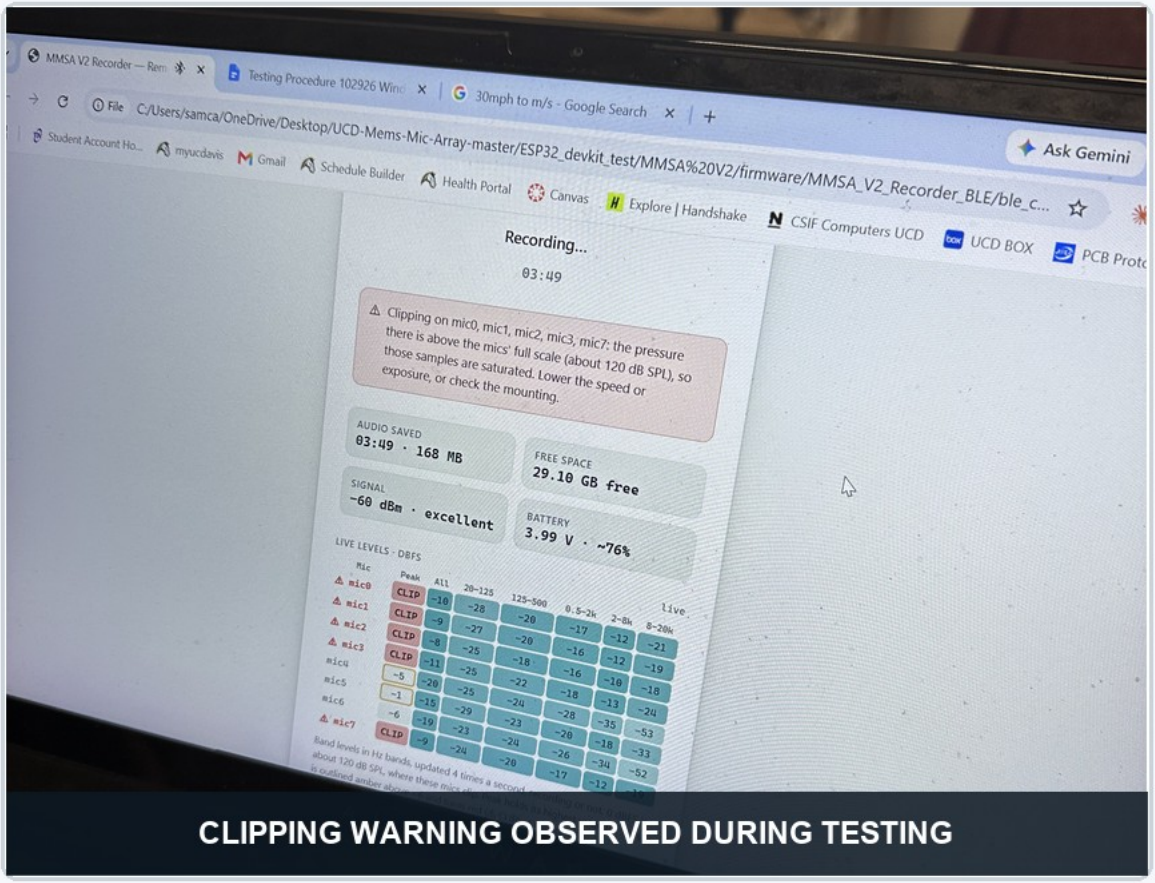

4-8 kHz: break moves forward with speed
Level (dBFS)
−15
−20
−25
−30
−35
−40
−45
−50
−55
30 mph, 4-8 kHz
60 mph, 4-8 kHz
0.00
0.25
0.50
0.75
1.00
0
1
2
3
4
5
6
7
x/c (0 = aft edge, 1 = forward edge)

Live monitoring also exposed the main 60 mph limitation

The recorder displayed per-channel, per-band levels and raised an explicit clipping warning during the run.



LIVE EIGHT-CHANNEL BAND-LEVEL TABLE



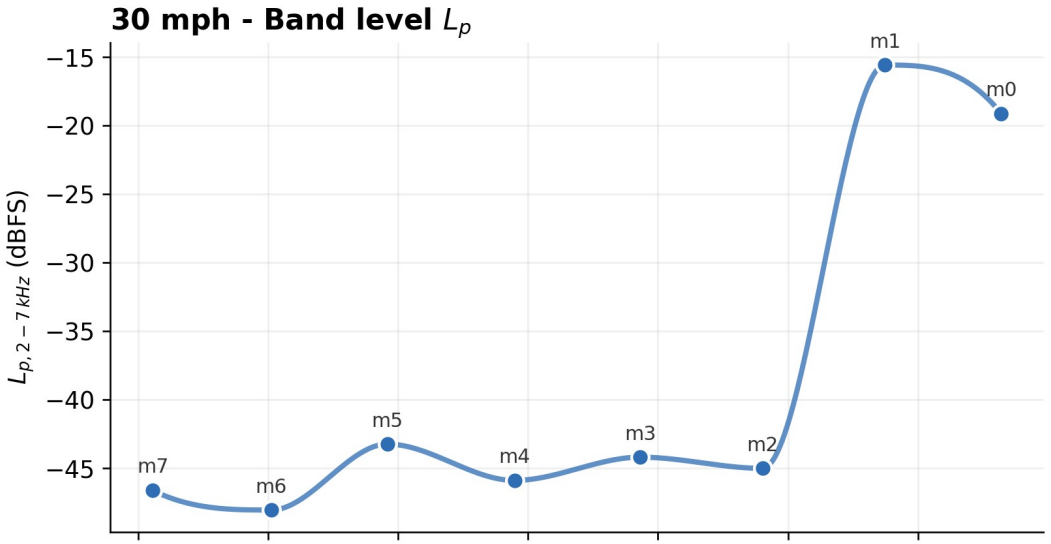
CLIPPING WARNING OBSERVED DURING TESTING

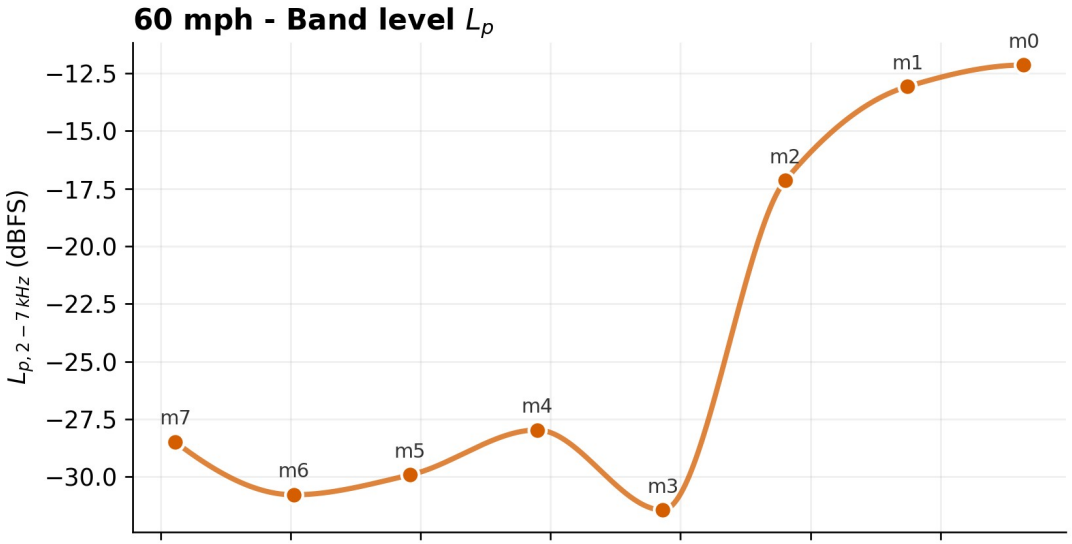
Analysis consequence: mic0–mic2 are clipped at 60 mph.

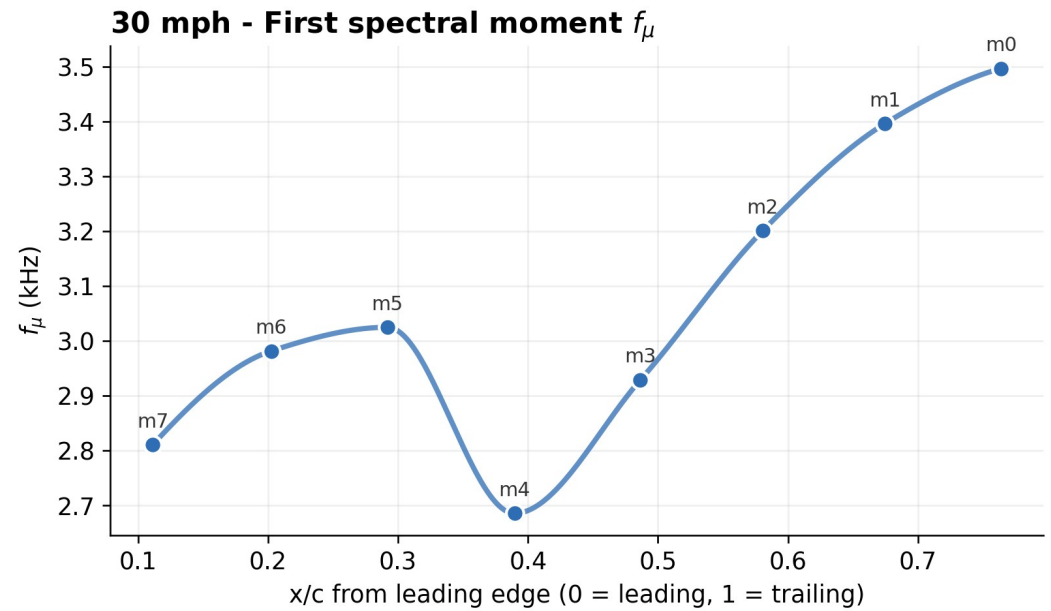
Use the 30 mph case as the cleaner quantitative result; treat 60 mph Lp and fμ magnitudes cautiously.

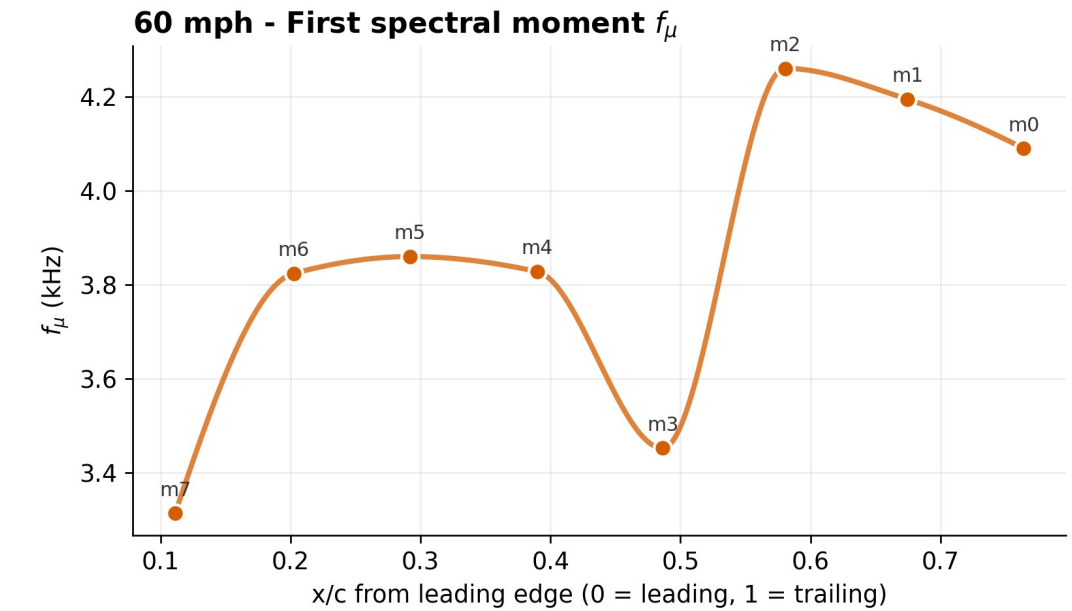
2-7 kHz pressure-spectrum metrics on the MMSA wing array

Dots are measured microphone values; curves are shape-preserving interpolation for visualization only.



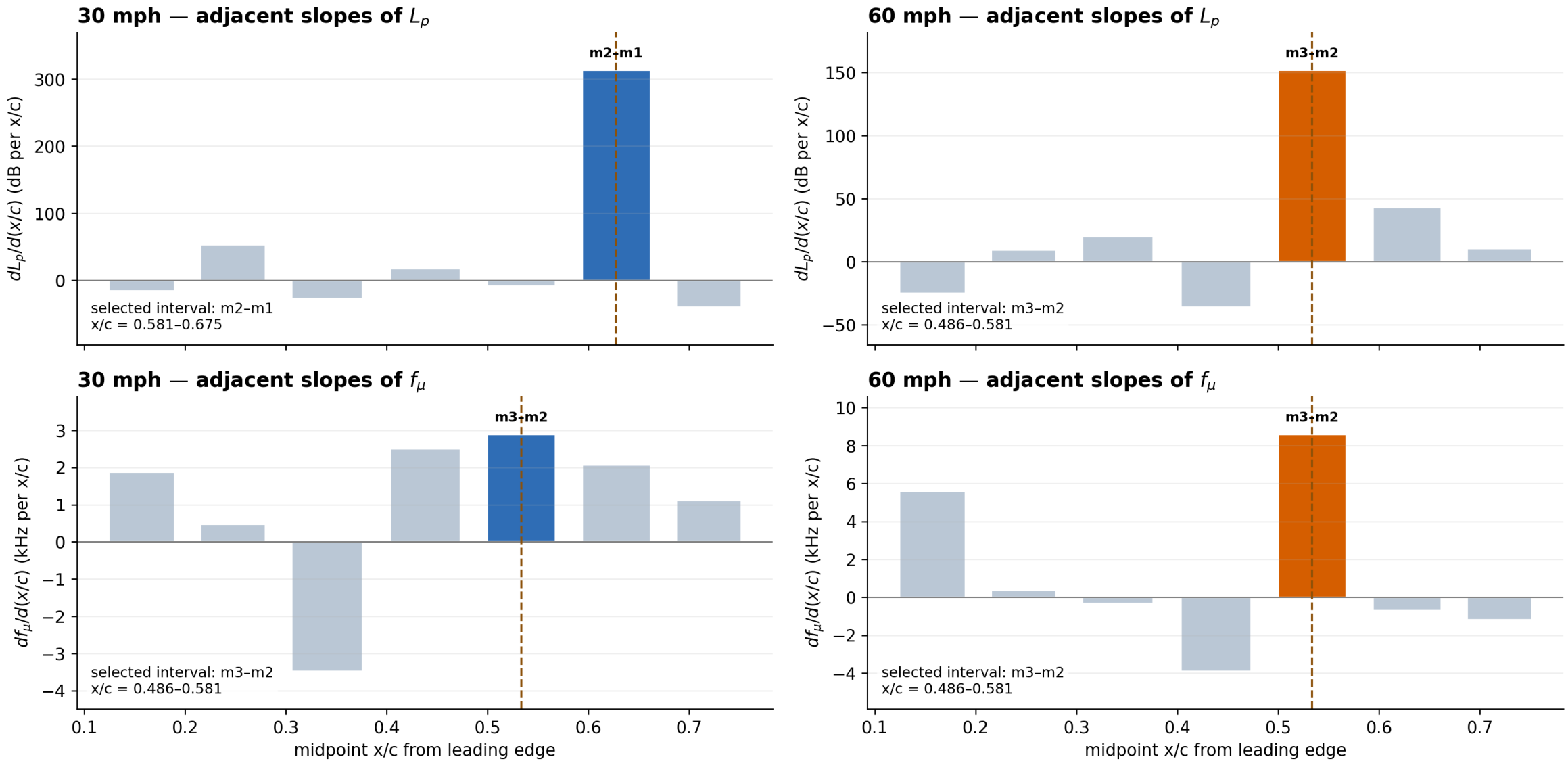






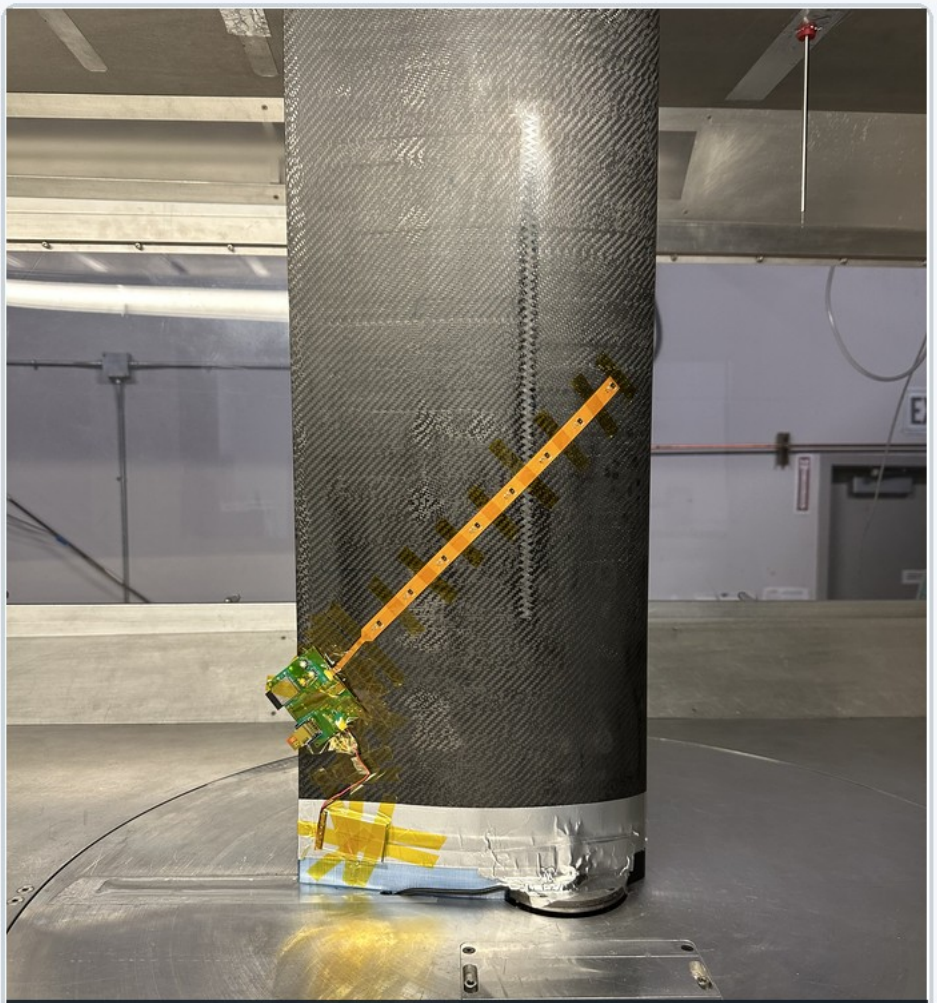
Paper criterion applied to MMSA: largest positive chordwise derivative

Bars are measured finite differences between adjacent microphones. The highlighted bar is the reported bracket; interpolation is not used to claim sub-spacing resolution.



What the wind-tunnel test established

The photographic reference and both acoustic methods identify the same microphone-scale region at 60 mph.



INSTALLED MMSA ARRAY IN THE TEST SECTION

4–8 kHz

produced the clearest spatial separation during the 30–60 mph ramp.

mic2–mic3

contains the 60 mph fluorescent-flow boundary and the measured acoustic level break.

x/c = 0.486–0.581

is the 60 mph paper-style Lp and fμ derivative interval supported by the mic spacing.

Report this as an acoustic transition estimate—not independent proof of boundary-layer state.

Next steps toward deployment

The core recorder works; the remaining work is board bring-up and application-specific validation

1	**Define deployment**	Replace the use-case placeholder with the exact study, placement, duration, and channel requirements.
2	**Bring up ordered boards**	Verify power rails → USB/flash → PDM clock/data → SD → BLE → multi-mic channel map.
3	**Connect the flex array**	Measure clock integrity, cable behavior, channel pairing, and simultaneous capture limits.
4	**Package the system**	Finalize mounting, enclosure, strain relief, acoustic opening, and environmental protection.
5	**Run a controlled pilot**	Use repeatable geometry, metadata, timing, and a validation recording before field deployment.

Main message

The project is now a working recording platform—not just a microphone breakout—and its reliability problems have defined the final hardware and firmware architecture.